

Brushless Carwash

The Three

Problem Statement:

The goal of this project is to successfully transform .pdc data of a car into an accurate 3D mesh which will then be exported as an STL file that will be used for a brushless car wash, ensuring important details such as mirrors, tow bars and bull bars are properly identified and included.

This will be achieved by a series of steps in a standardized pipeline that will receive the raw .pdc data, removing any detail that is not in reference to the car. We consider the “noise” in following:

- Floor
- Ceiling
- Side walls
- Below curtain
- Statistical outlier (Points that are significantly distant from neighbors)
- Isolated point clusters (Small groups of points disconnected from the main point body)

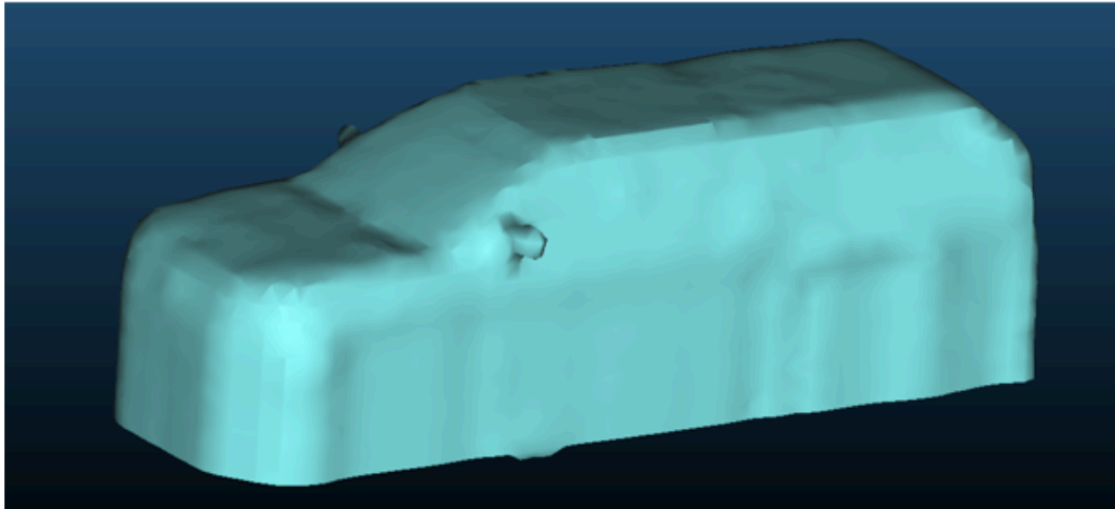
We highlight important parts of the car such as:

- Wheels
- Mirrors
- Roof of the car
- Front extreme (bull bar)
- Rear extreme (tow bar)

Which then will be transformed into a mesh that will be used to safely wash the car automatically.

The end POC: Working program/converting software

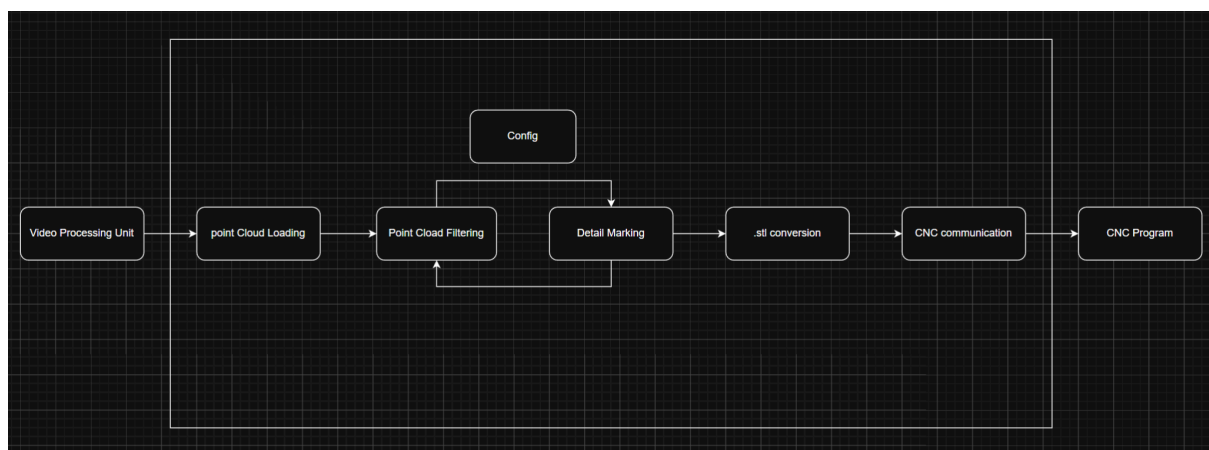
Success criteria: We aim to reach results like pictured here, but more precisely and as detailed as possible (detailed mirrors, bull bar ...)



Work plan:

1. Extract raw point cloud data(in .pcd) from the OVP-based camera system
2. Filter noise and erroneous measurements
3. Detect and preserve critical vehicle features
4. Identify and handle abnormal or unsafe data conditions
5. Convert the processed point cloud into a watertight, high-fidelity 3D mesh
6. Export this mesh as a single STL file suitable for downstream path-planning algorithms

The proposed flow:



Research techstack:

- Language
- Libraries
- Extensions
- Applications to work with

Week 1 – Project Progress & Discoveries

In the first week of the project, our main focus was understanding how to work with 3D point cloud data. This was completely new for us, so a lot of time was spent exploring how point clouds are structured and how they can be processed using Python and Open3D.

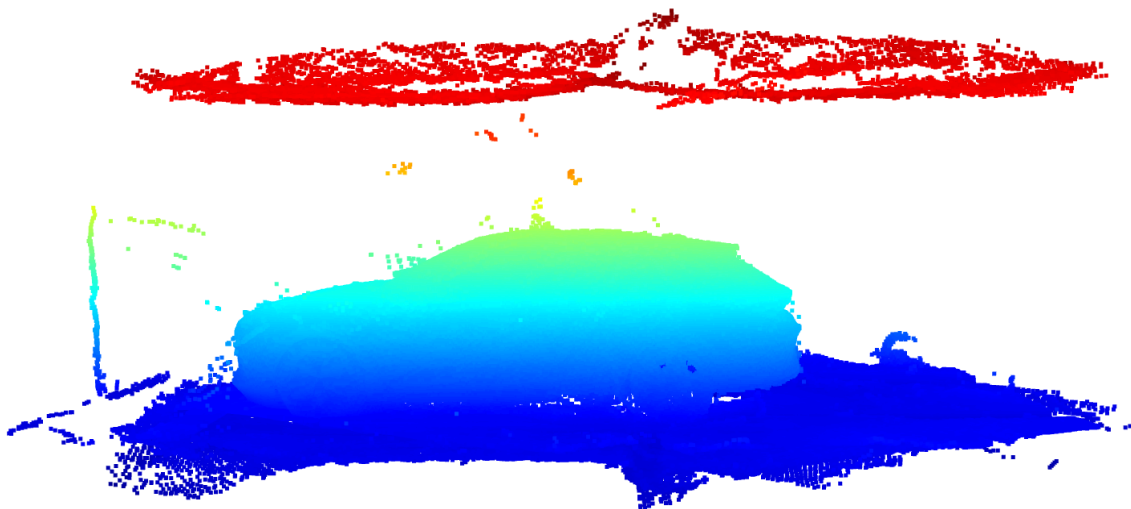
We learned how to load and visualize point cloud data and quickly discovered that raw data contains a lot of noise and unwanted elements, such as the floor and ceiling. Initial attempts to remove the floor using simple height thresholds were not very reliable. This led us to research more advanced techniques like plane segmentation using RANSAC.

Through experimentation, we discovered that detecting planes works well, but removing them correctly is more challenging than expected. We learned that downsampling improves stability for plane detection, and that checking the plane's orientation (using normals) is important to distinguish the floor from walls or other surfaces.

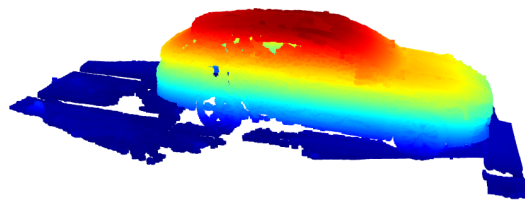
Overall, this week was mainly about exploration and learning. While the solution is not yet fully robust, we now have a much better understanding of point cloud processing and the challenges involved. This forms a solid foundation for improving accuracy and robustness in the coming weeks.

Example 1:

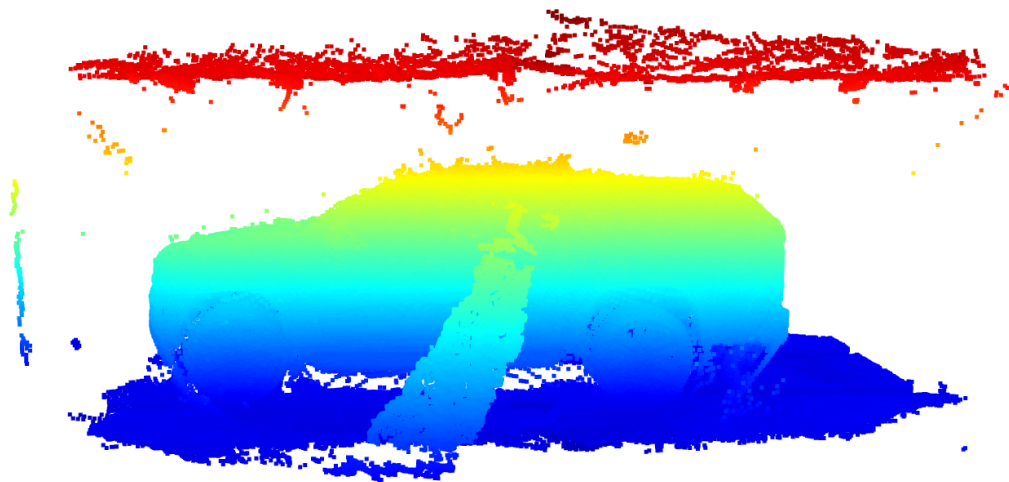
Original Point Cloud Data file:



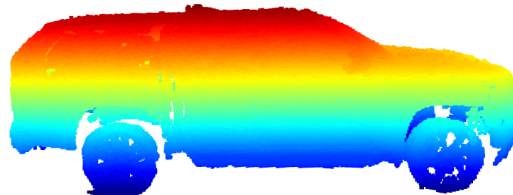
Detected and removed ceiling, yet the floor is partially removed.



Example 2: Amarok met schuim
Original Point Data Cloud:



Main Car cluster after detecting floor and ceiling, and filter them. In this example it worked better.



Week 2 :

The goal of this week was to understand the structure and quality of raw point cloud data coming from the OVP-based camera system, and to make the concept of “noise” and “invalid measurements” concrete and visible.

This week we focused on exploring a single raw .pcd file of a car with foam applied. Since this data was new to us, the main focus was not on producing a final mesh yet, but on **understanding what is actually inside the point cloud**.

We implemented a first-stage diagnostic script that:

- Loads and inspects the raw point cloud
- Analyzes the coordinate system and bounding box
- Detects large planar surfaces such as the floor
- Categorizes every point into meaningful groups
- Identifies important car features
- Produces both visual and textual diagnostic output

Findings and discoveries

1. Raw data contains a lot of non-car data

From ~922,000 points, only ~42% belongs to the actual car. More than half of the data consists of floor reflections, side walls, noise, and measurement artifacts. This clearly shows why raw data cannot be used directly.

2. “Noise” and “invalid measurements” are different

We learned to distinguish:

- **Invalid measurements** (floor, walls, below-curtain points) caused by the environment or setup
- **Noise** (statistical outliers and isolated clusters) caused by sensor instability, reflections, or foam behavior

By visualizing these categories with different colors, the difference becomes clear for both developers and customers.

3. Floor detection is critical

The floor alone accounted for more than 50% of all points. Detecting the lowest horizontal plane proved to be a reliable way to identify and classify floor data.

4. Important car features are detectable early

Even before mesh generation, we were able to detect and highlight:

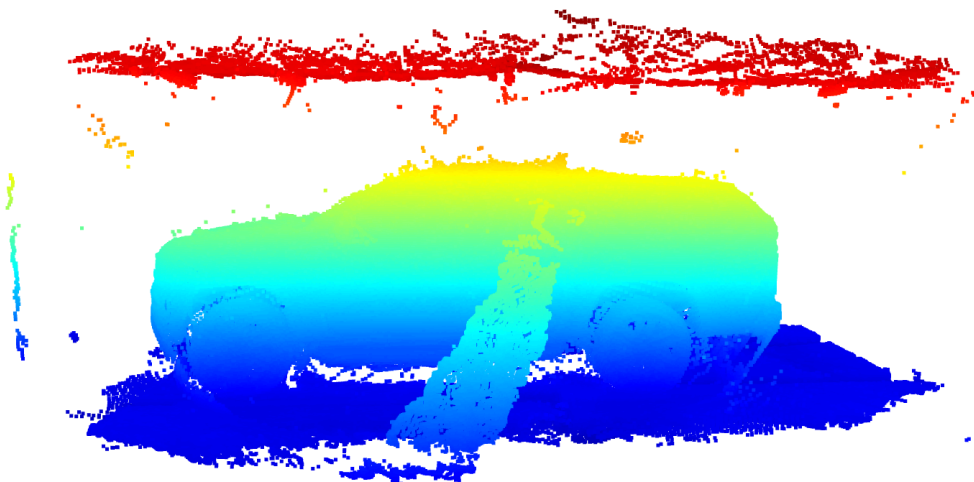
- Wheels
- Mirrors
- Roof area
- Front and rear extremes (possible bull bar and tow bar)

This confirms that feature preservation must already be considered in early filtering stages.

5. Dimensions become reliable after cleaning

After removing noise and invalid data, the measured car dimensions were realistic and within configured limits. This validates the cleaning approach.

Original point data cloud:



The program detects:

Total points: 922,624

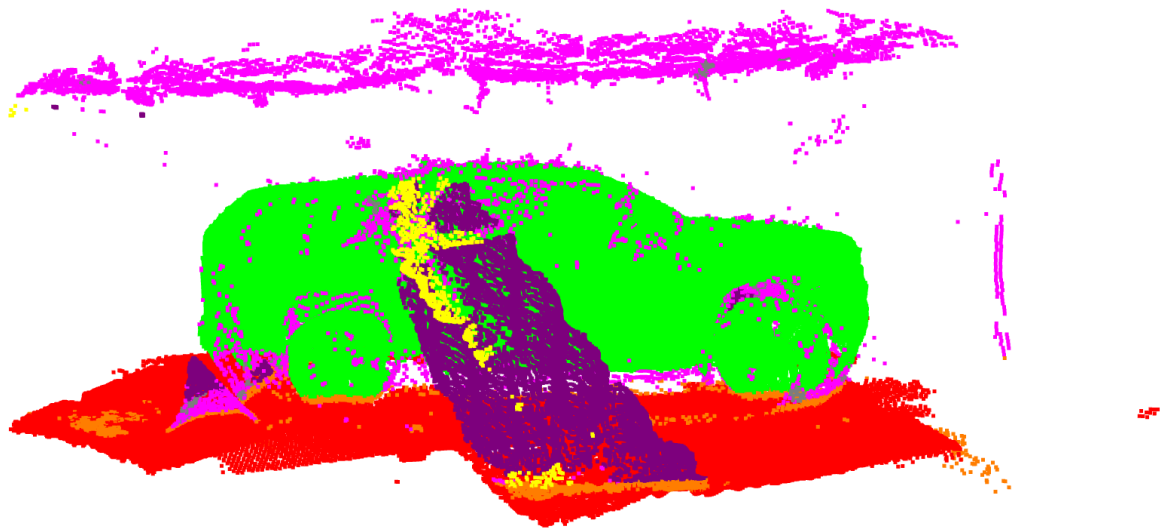
Bounding box:

X: -0.331 to 8.667 m (range: 8.998 m)

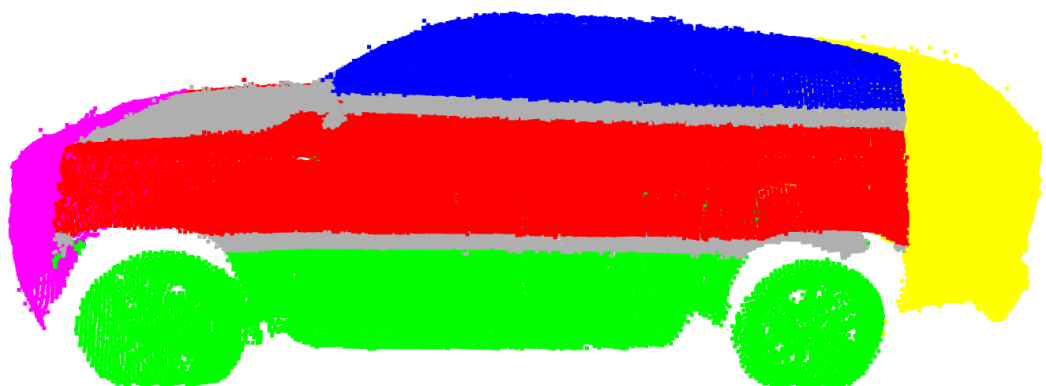
Y: -0.107 to 5.105 m (range: 5.212 m)

Z: -0.381 to 3.025 m (range: 3.406 m)

- **Data quality analysis - color coded :**



Car features highlighted:



● Green = Wheels

- Red = Mirrors
- Blue = Roof area
- Yellow = Rear extreme (tow bar)
- Magenta = Front extreme (bull bar)

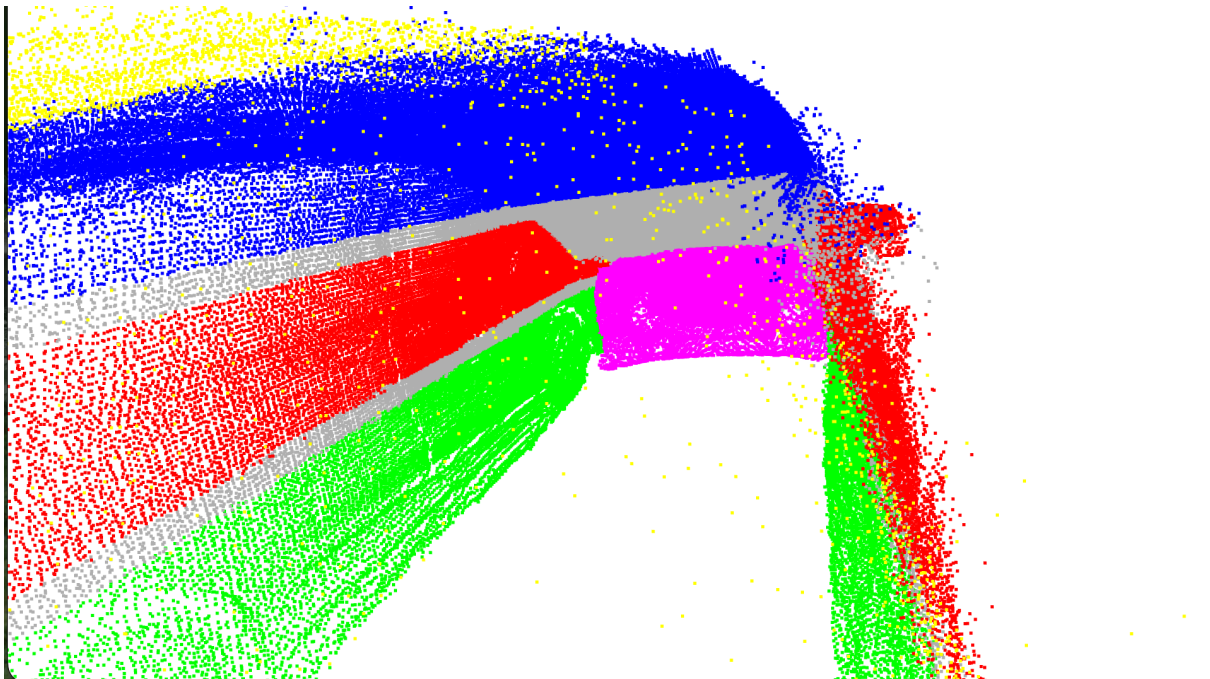
Week 3-5:

We've created a config.yaml file where customer can tune the parameters to change the configuration of filtering mechanism, here is an example:

We have these variables by default, based on results of filtering we can tune to make filtering more aggressive and efficient in removal of noise points(have to know the balance tho, because if these parameters are too high, the filtering becomes too aggressive)

With these parameters, the filtering worked not so well, so we have to tune:

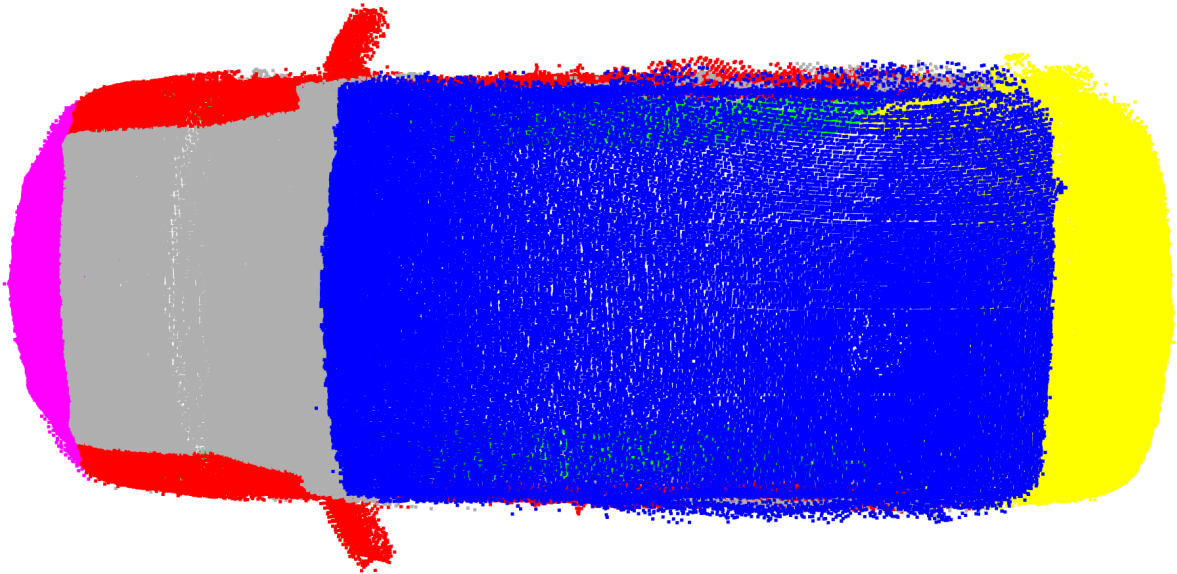
```
noise_detection:
  # Statistical outlier removal
  outlier_neighbors: 100      # Number of neighbors to analyze (recommended: 15-30)
  outlier_std_ratio: 1.3      # Standard deviation threshold (recommended: 1.5-3.0)
```



As we can see we have these extra points on the side of the car, they weren't filtered, so we can tune the parameters mentioned above to reach a better result:

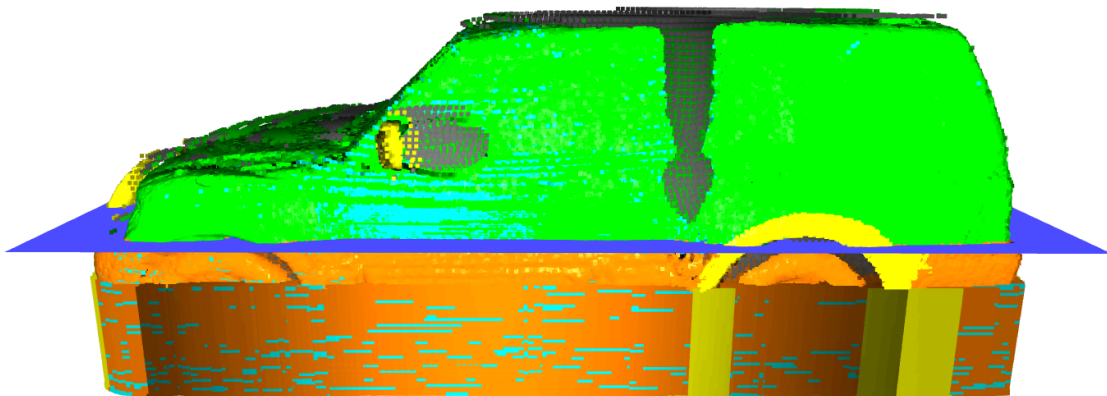
```
noise_detection:
  # Statistical outlier removal
  outlier_neighbors: 140      # Number of neighbors to analyze (recommended: 15-30)
  outlier_std_ratio: 0.9      # Standard deviation threshold (recommended: 1.5-3.0)
```

And as you can see, on the right side of the car there are much less noisy points.



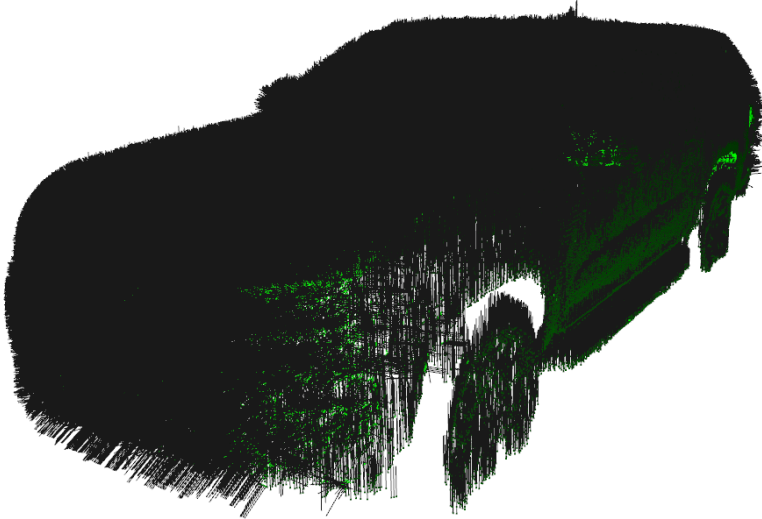
The client asked for the curtain to be from the top part of the wheel down to the floor.

Firstly, we had to detect the highest Z point of the wheel area, note the height in the report and draw a plane there to put the curtain down from that point:



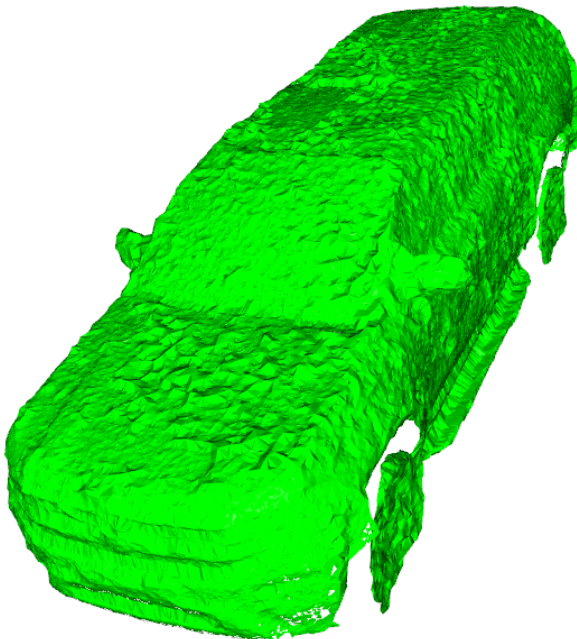
After the first week i got to this result

After using rufina's cleaned cars I started looking into meshing algorithms. Apparently we needed to add normals to the points in order to create meshes. This wasn't all too difficult. (black lines)

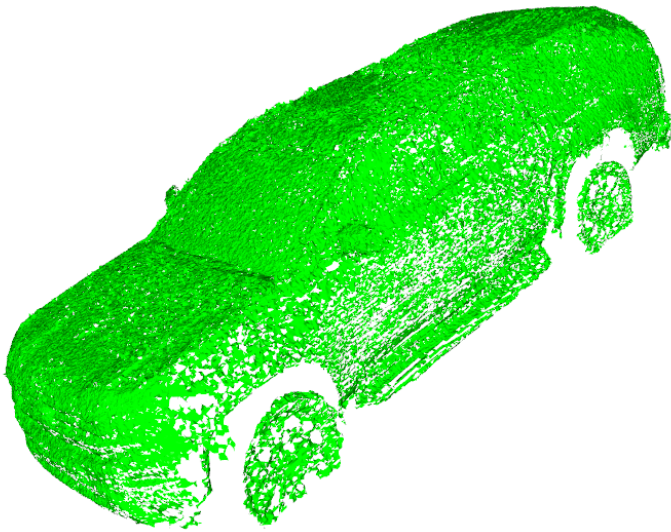


So now time to try out some meshing algorithms:

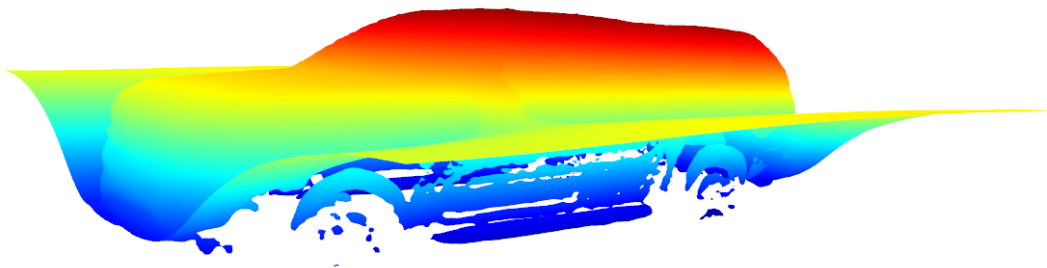
Alpha:



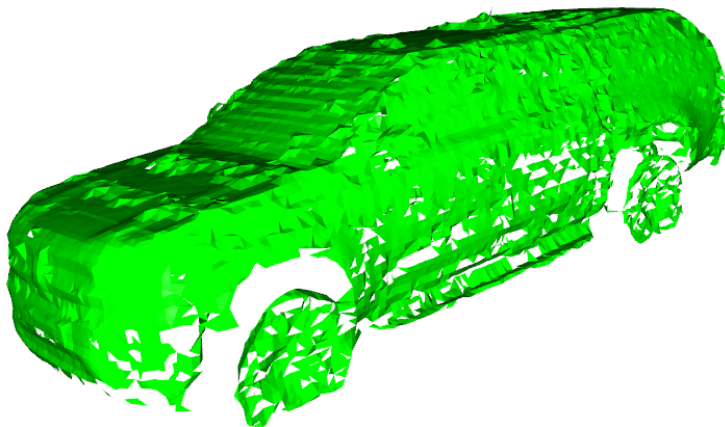
Ball Pivoting:



Poisson:

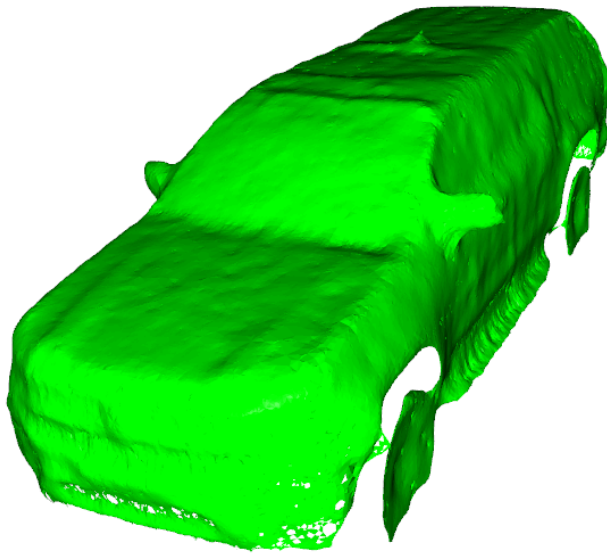


After this I continued experimenting a bit with the ball pivoting since it was better at catching details than alpha but never got it to be nice without holes, even after downsampling



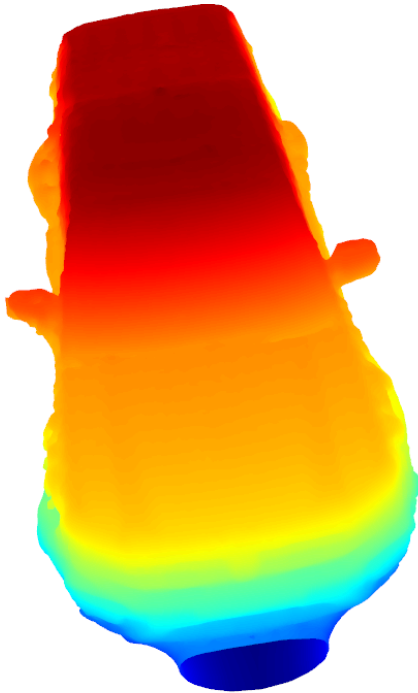
So I decided to continue with alpha reconstruction for now.

Seeing as it looked a bit rough I decided to apply some smoothing with smoothing filters. Averaging filter would make the car ever so slightly smaller but gives a smoother look then laplacian filter so i decided to to use the averaging filter.

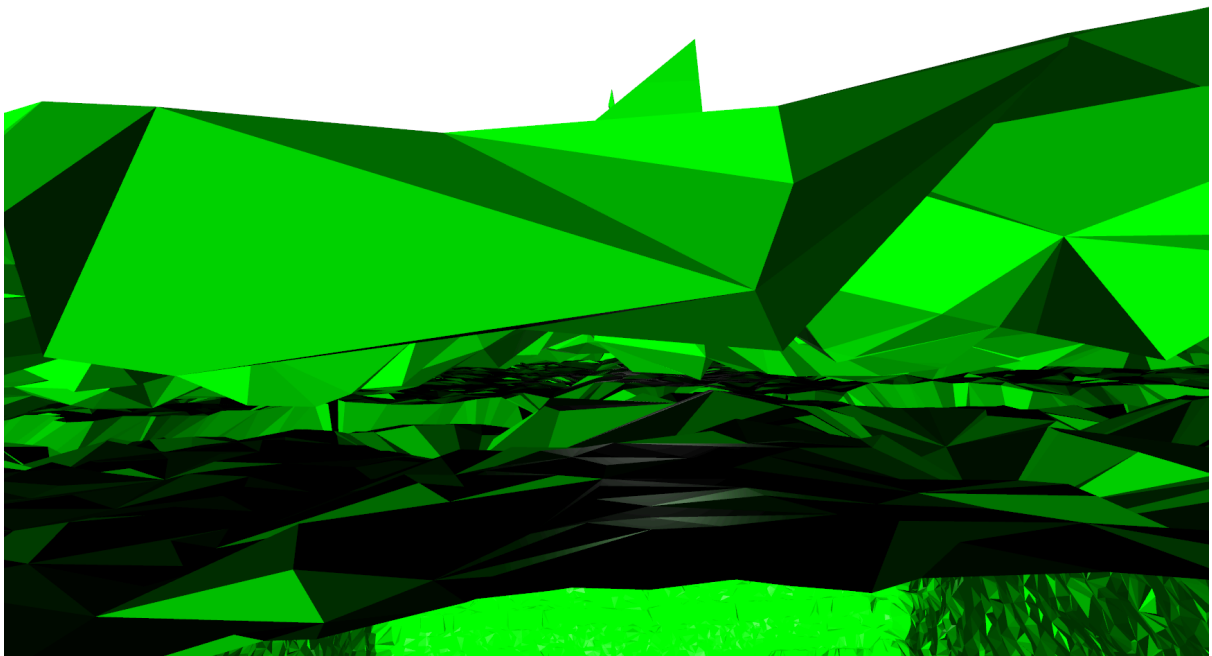


All of the sudden I noticed the normals were a bit weird, if we look back we can see that on the sides the normals are pointing upwards instead of outwards, after making them point

outwards i got this result with poisson so I started trying out both a bit:



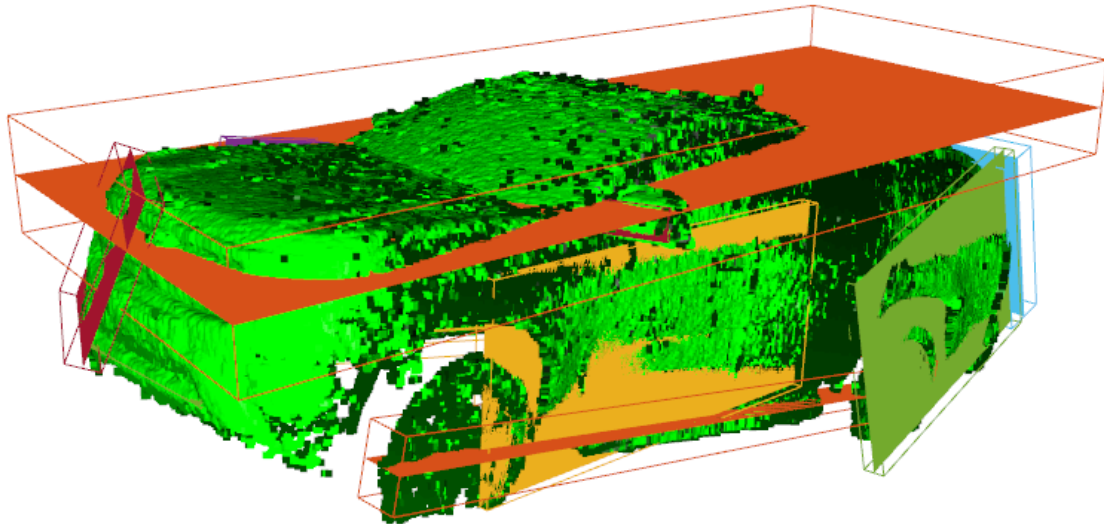
After looking closer at the alpha shapes (and a bit at the ball pivoting) I noticed that we were creating a sort of double shell where our shell was not just 1 layer but actually 2 layers making it a full object. If we zoom into the roof we can see the following:



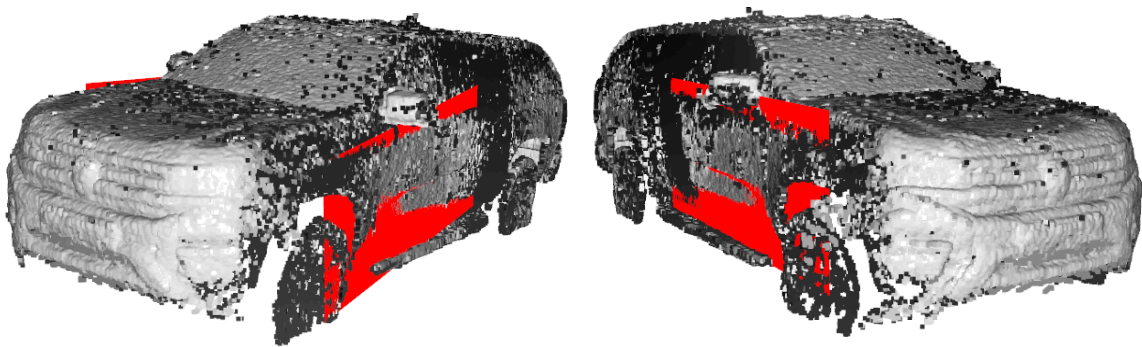
After discovering this and noticing that poisson does not have this problem I decided to only continue with poisson from now on.

Now I wanted to try and get rid of the weird bulge on the sides of the car so started experimenting with cutting off the sides and cleaning them more separately.

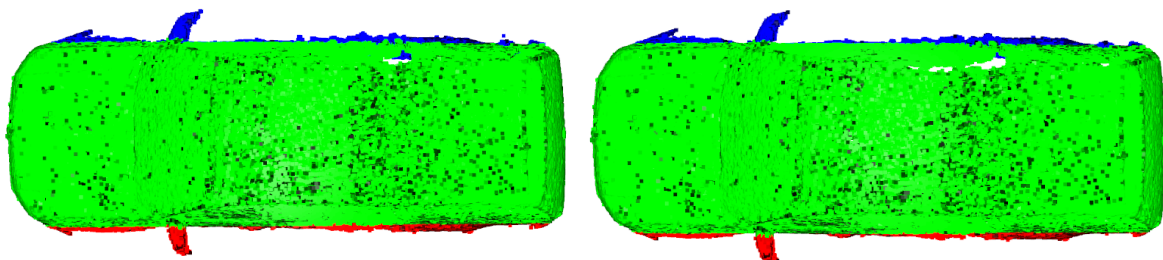
To do this I wanted to experiment with finding the side plane of the car so I could take it off, clean it, and put it back on. Here is a result i got for cutting of the sides of the car:



After filtering for the 2 planes with the biggest y component (sideways) I ended up with these 2 planes:

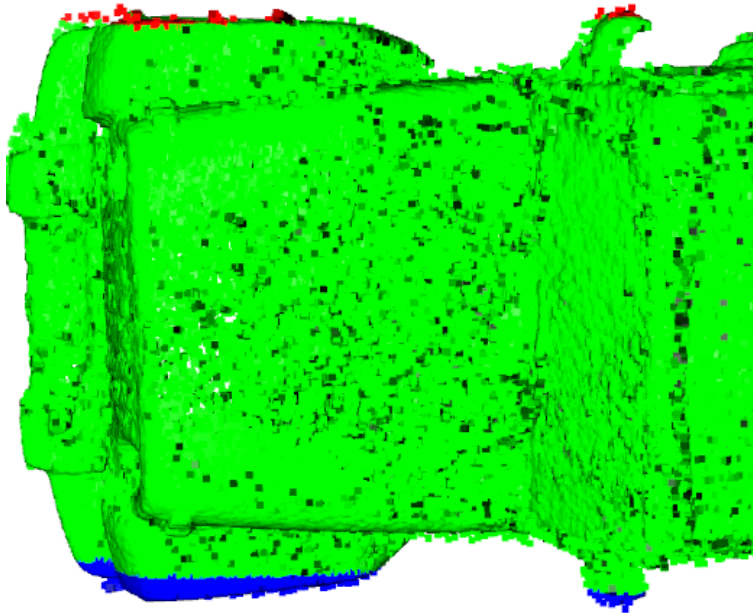


After separating the sides and cleaning them we get this result (with some painting to indicate sides):



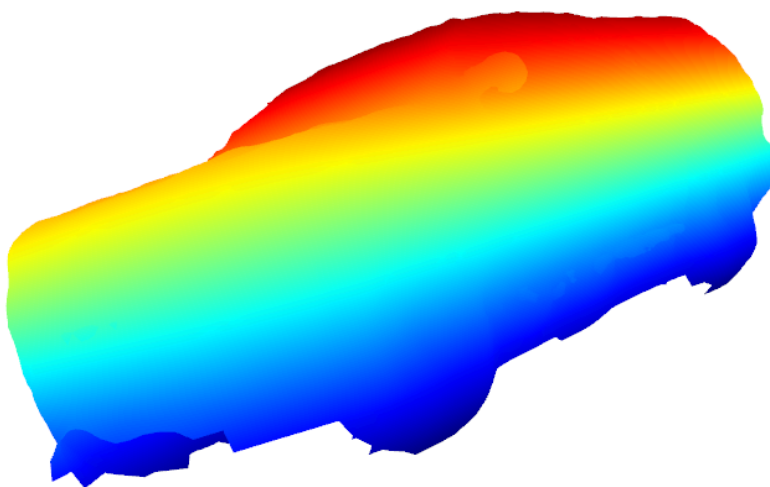
And after reconstruction the bulge is still there, just a bit smaller.

After running this on a different car (which is unlikely to pass by often) i decided to not use it anymore since it didn't even capture the mirrors or doors (which i wanted to capture):

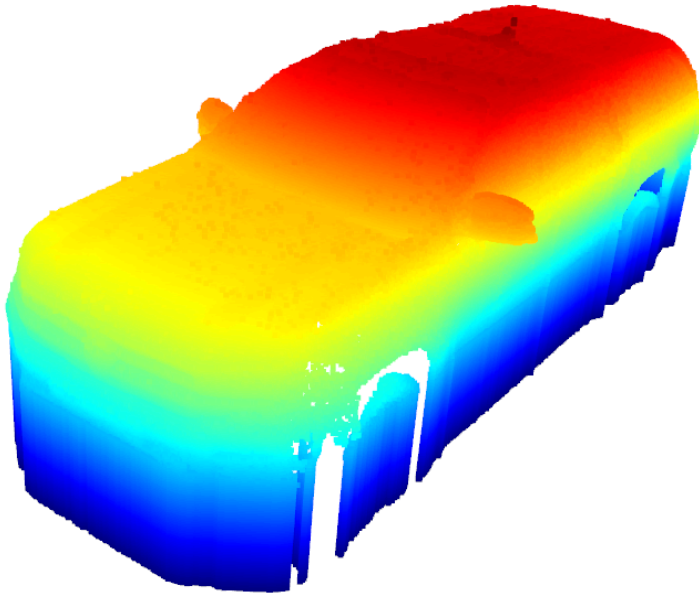


After some more looking into cleaning and trying out meshlib instead (paid library) I noticed that when using the meshlib normal estimation the bulge I had before wasn't there anymore so I started looking into different ways to estimate the normals with open3d. After a bit of work I got a similar result without the bulge.

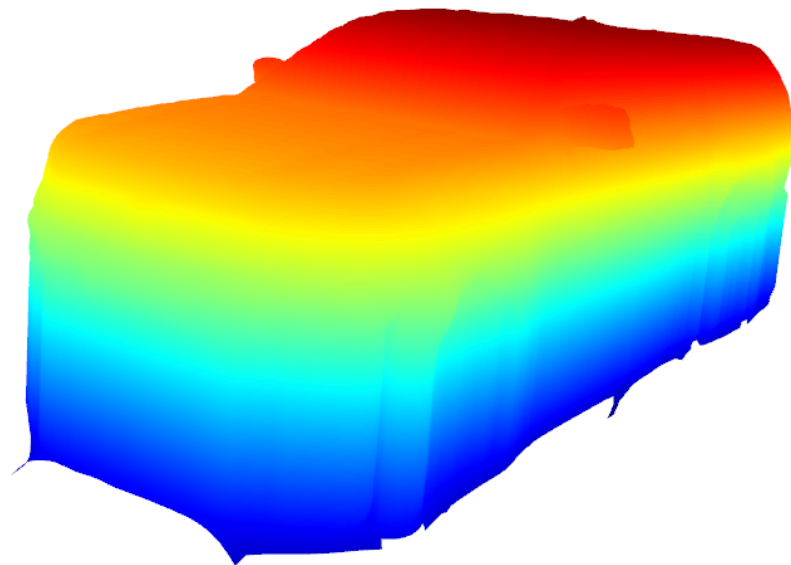
Now I get rid of the curve poisson adds to close the shape to have an open bottom:



Now I started looking into making the sides drop down to 0 so we have a nice “curtain” going down to the floor.

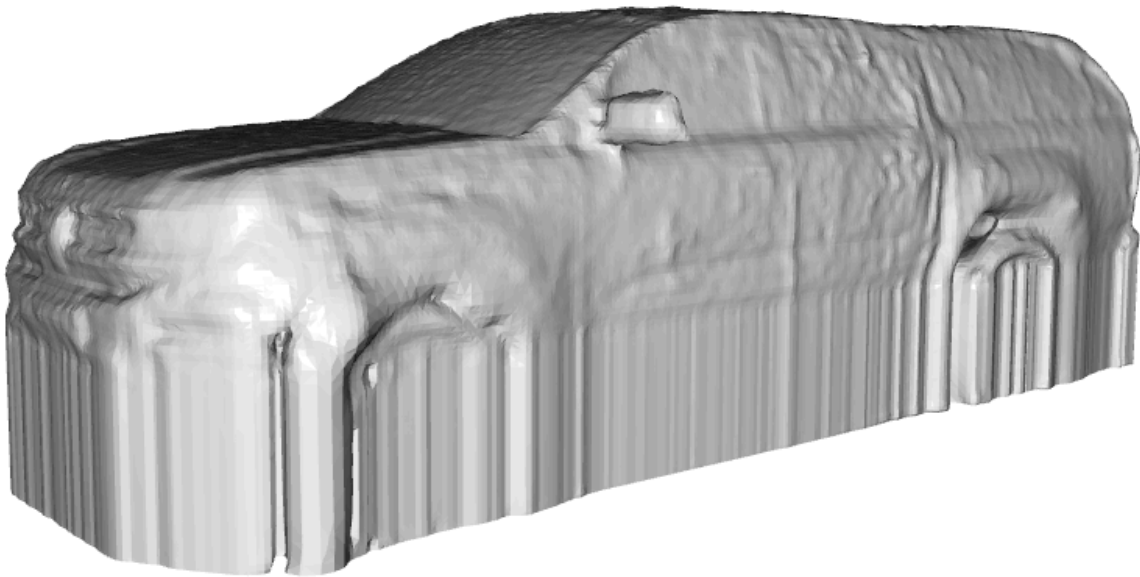


Using the reconstruction and opening the bottom we get the following result:

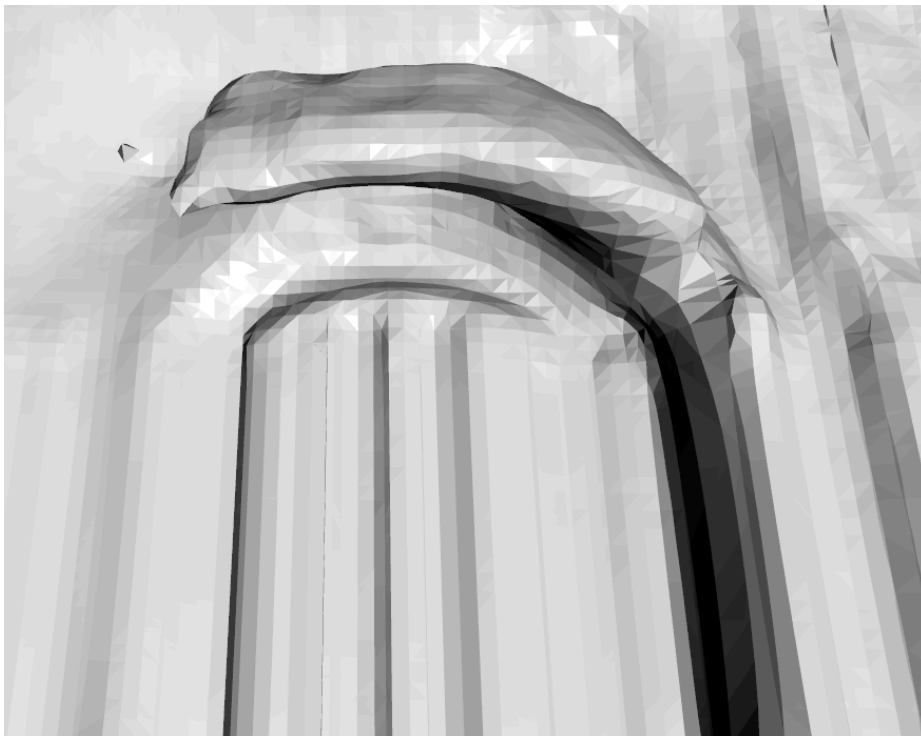


This however is not a nice flat bottom and not all at $z=0$ so I looked into cutting it instead of deleting “uncertain” parts. At first I ended up with a jagged edge but after changing to cutting the triangles (and replacing them with smaller ones) instead of removing all below $z=0$ I got

this result:



This might look good on the outside and especially the top part but we still got a lot of weird stuff happening in the wheel wells:

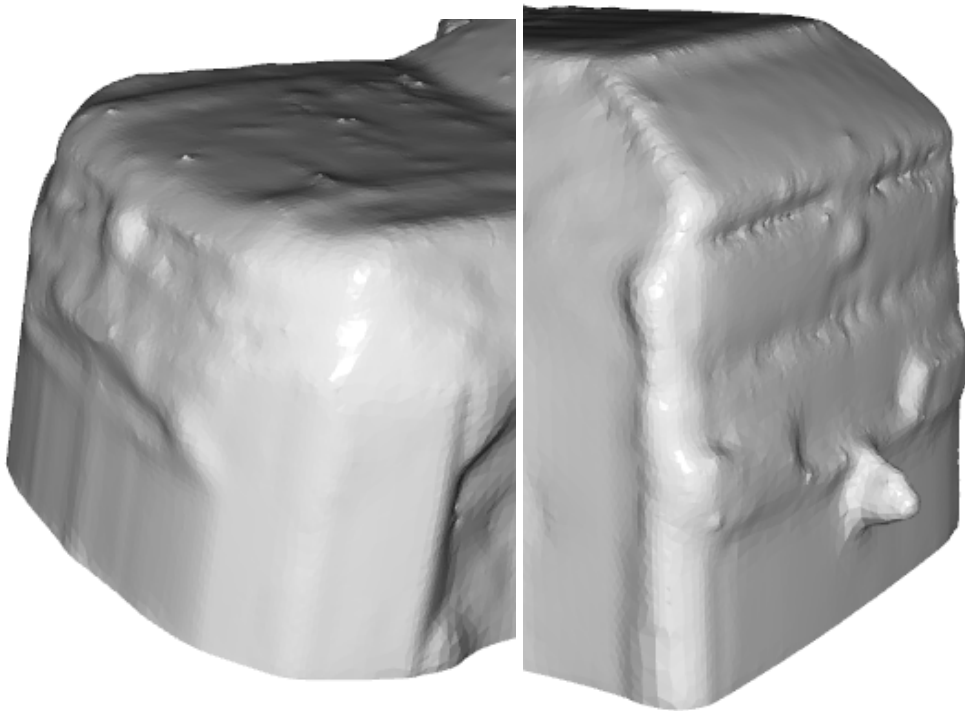


To combat this I started looking into getting rid of anything on the inside of the car. I ended up using a method where i place 8 cameras on the outside of the car and if a point is detected by 1 of the cameras (not covered by another point with a certain radius around it)

and the weird inside artifacts were pretty much gone (picture is taken from the bottom):

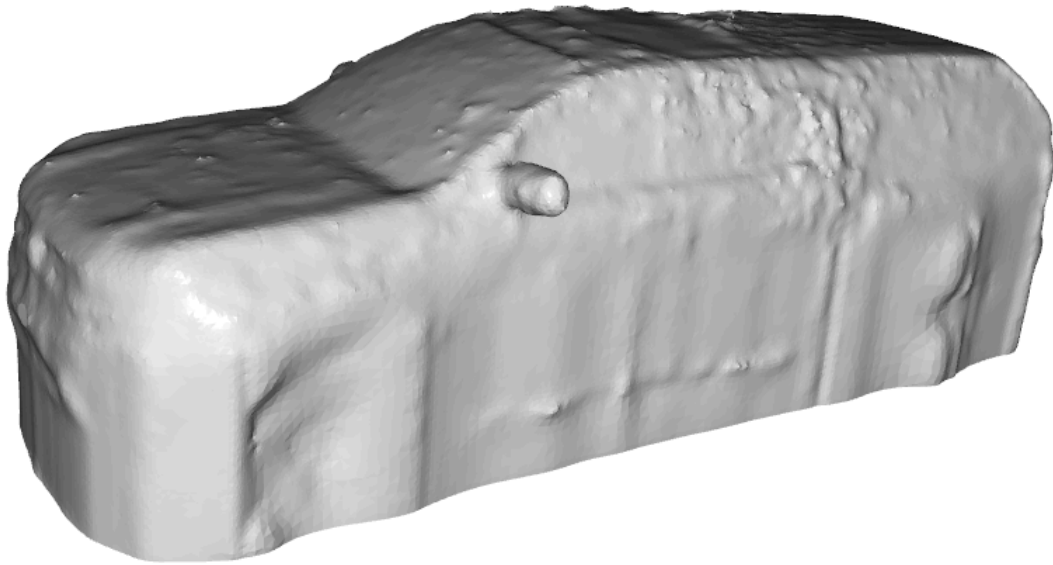


After experimenting with keeping the car whole, then adding the curtain as an extra and removing the interior points after adding the curtain I managed to preserve the bumper and towbar instead of getting rid of them.



This was the final result after all the testing but I do believe it could be improved by fine

tuning the parameters better:



Foundation & Technology Stack

The project is built on Python with Open3D as the primary 3D processing library, complemented by NumPy for numerical operations, SciPy for spatial algorithms (cKDTree, ConvexHull, distance_transform_edt and gaussian_filter), Shapely for concave/convex hull geometry, and PyMeshFix for watertight mesh repair. Open3 was eventually decided on as the most useful as it provides native point cloud + mesh support with Poisson reconstruction built in, unlike trimesh (mesh-only) or PCL (C++ overhead). SciPy was chosen over scikit-learn as cKDTree is faster than sklearn's BallTree for 3D nearest-neighbor queries, and SciPy offers EDT/Gaussian smoothing that sklearn lacks entirely.

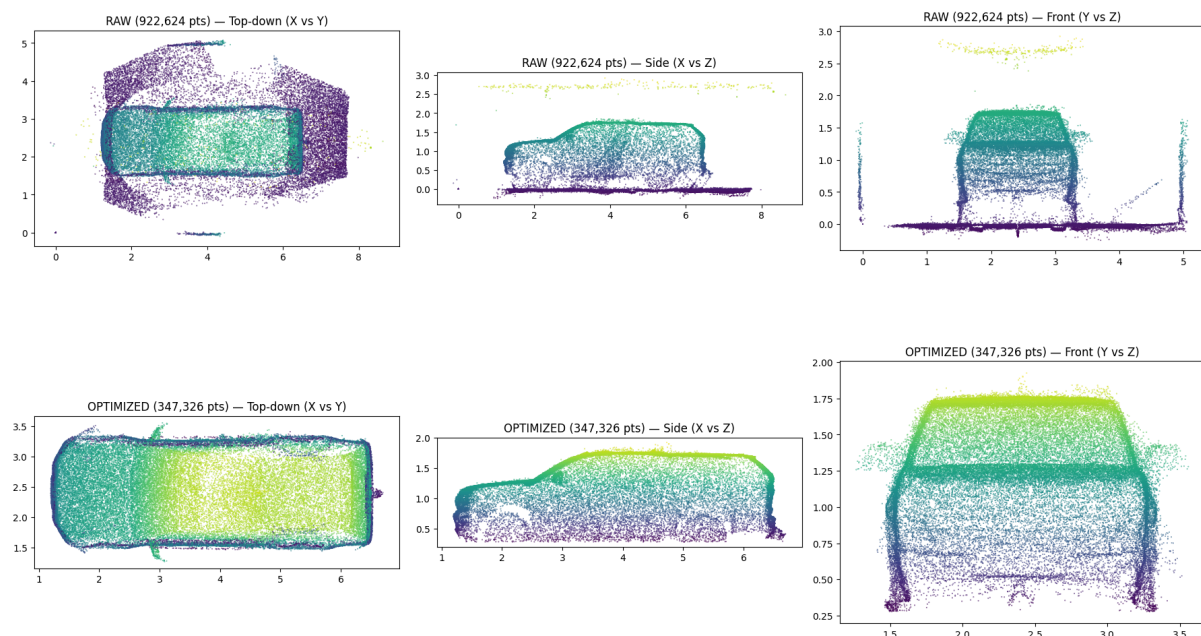
Building the Cleaning Pipeline

The cleaning pipeline (from the carewash_pipeline repository) has 11 stages. Stage 1: voxel downsample (0.008m) It is used as it reduces 1M raw pts to ~400k while preserving density uniformity. Stage 2: normal estimation which is needed for floor detection. Stage 3: RANSAC plane removal, it removes floor/ceiling/walls. (Later we also implemented a box margin check to avoid removing car body planes. You can see in the following figure how without these checks, the walls are removed). Stage 4: normal-based floor filter (upward normals below 0.35m) This catches anything that was not caught from the other filters related to the floor. Stage 5: Z-crop (-0.05 to 2.0m these constraints are set due to the actual places dimensions) and always catches the roof that is almost present in every scan (note for future work and testing, perhaps this could be removed as clustering would probably identify this and remove it). Stage 6: statistical outlier (nb=10, std=3.3) This was kept as it

usually only removed small things around important areas (mirrors, front and rear). Stage 7: radius outlier (nb=5, r=0.05m) this further helps and works very similarly to the statistical outliers in actual use. Stage 8: DBSCAN clustering (eps=0.10m, min=20; largest cluster = car). Stage 9: raw recovery it re-samples original raw points within 0.001m of guide cloud to restore detail lost in voxel step (this is pretty useful for keeping as much mirror, our client put an emphasis on preserving mirrors and there are many things done throughout the project to closely match it to the raw image we get of the mirrors) It is also very important to note that there are limitations that are affected on the quality of the car scan, if its a bad scan of the mirrors, there is not a lot that can be done to successfully achieve this. Note for future work, there could be more strategies implemented to successfully achieve a mirror placement on bad data but due to time constraints this was not looked into. Stage 10: post-recovery floor filter. This adds a floor to close the car or else watertight flag would always come out as false. Stage 11: floor band removal (toggleable; skipped by default). In practice this would actually remove a part of the wheels (you can see this in some of the figures) so it was added as a flag that is set to off by default.

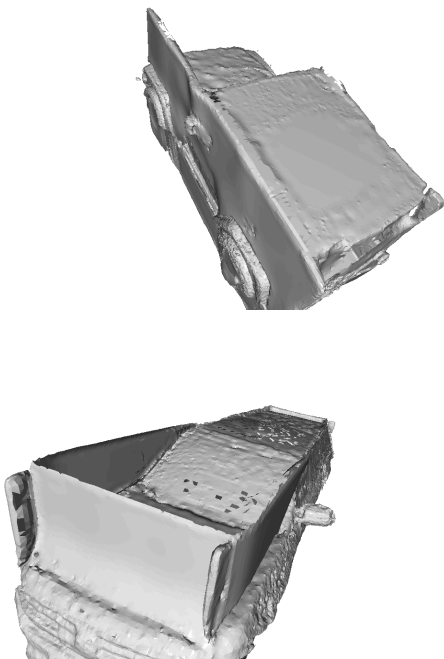
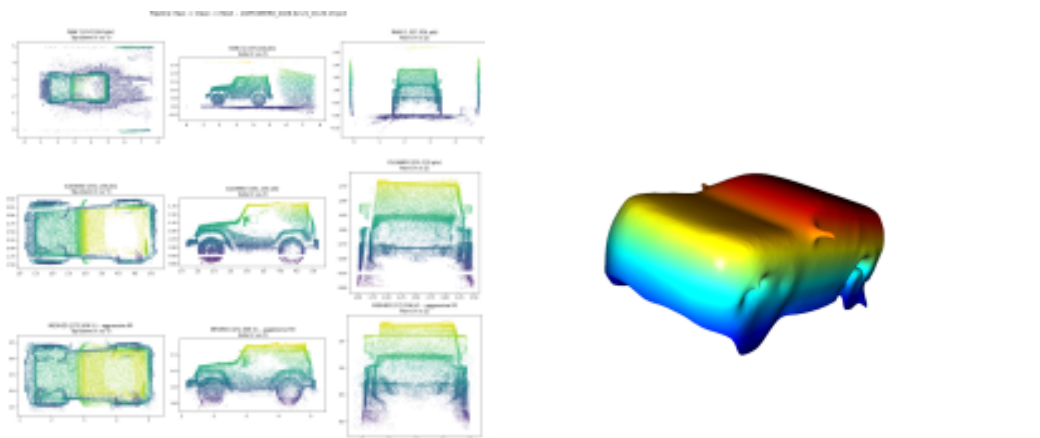
Another important factor to take into account is that Optuna was used for tuning to find optimal parameters. These values were later used for every other car with success. The parameters were tuned by doing systematic sweep experiments that would evaluate the point retention, visual quality and downstream mesh smoothness.

Raw vs Optimized — Amarak met schuim VGA.pcd

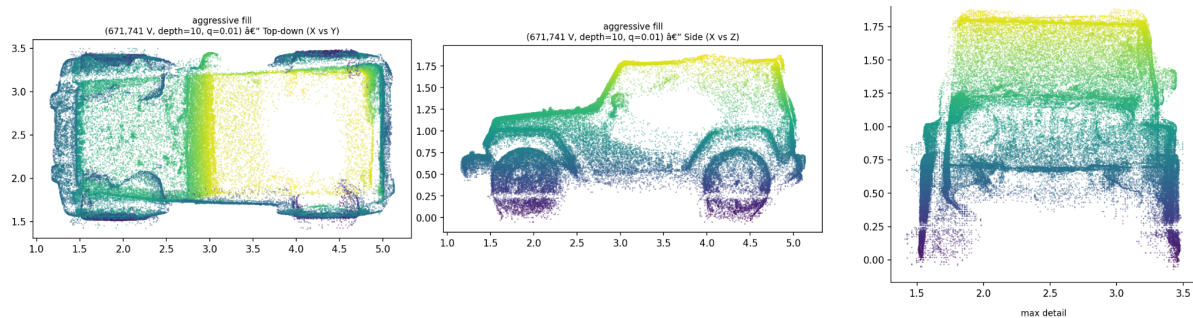


Meshing Algorithm Exploration Early meshing work tested multiple algorithms:

Three meshing algorithms were evaluated. Alpha Shapes: fast but produced double-shell artifacts (two mesh layers). Ball Pivoting: better surface detail than Alpha but always left holes even after cloud downsampling and it became unusable for watertight requirement. Poisson Reconstruction: chosen because it guarantees closed surfaces from oriented normals. Poisson is better for this challenge as it solves a global optimization (indicator function via octree), produces smooth gap-free surfaces. The trade-off is it adds geometry at boundaries (requires density trimming + box crop).



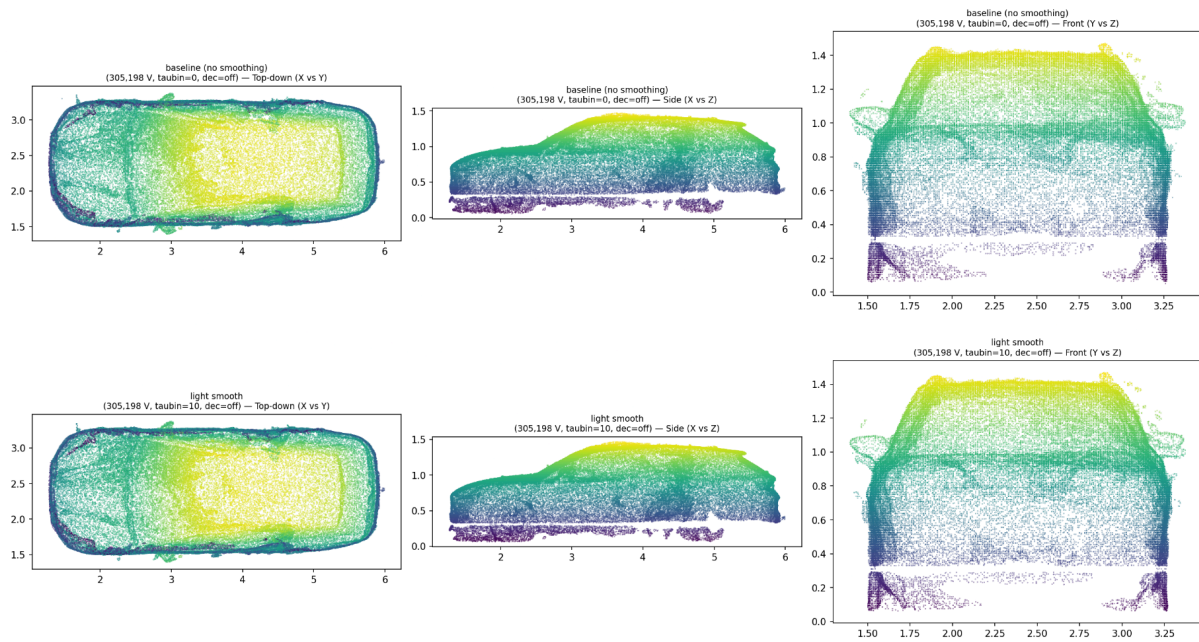
These figures show some of the testing, the first image shows some of the aggressiveness in the cleaning from the floor band removal (the wheels) it also shows that walls are removed from the planes without bounding box checks. The other figure is showing the effectiveness of smoothing as our client wanted something similar to this figure but the mirrors were severely damaged due to the smoothing. Another issue that was fixed by identifying corresponding points in the x and y axis in relation to the car was the creation of these walls that would stand out of the car. The same issue was also found in the bumper and rear.



Smoothing Strategies & Surface Refinement

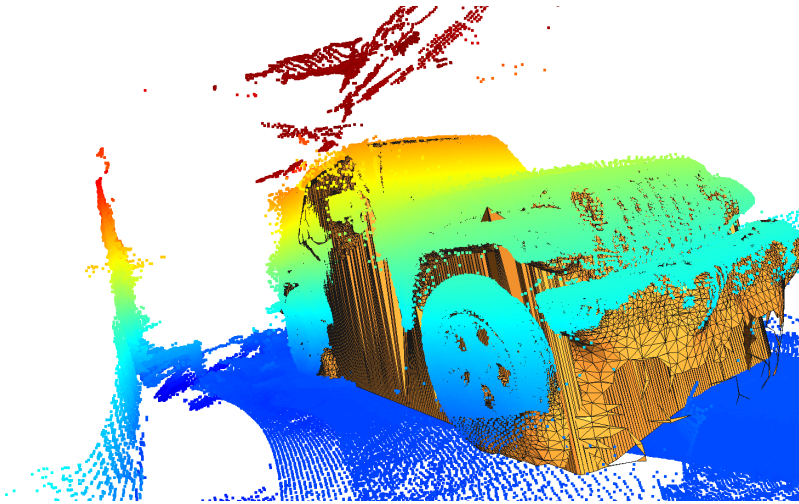
Some smoothing fails on dense meshes: Taubin/Laplacian smoothing moves vertices based on local neighbors. On 530k-triangle meshes, each vertex has many neighbors that all carry scan noise and the average of noisy neighbors is still noisy (points that are unnecessary to the detrimental figure of the car). Decimation solves it; it uses quadric edge collapse that forces vertex positions onto simplified planes, effectively low-pass filtering the surface. HC Laplacian then refines transitions. HC is more useful over standard Laplacian: HC adds a back-projection step that prevents shrinkage which is critical for preserving car dimensions. Pinning mirror/bumper vertices during HC preserves feature fidelity

Smoothing & Decimation — Parameter Comparison

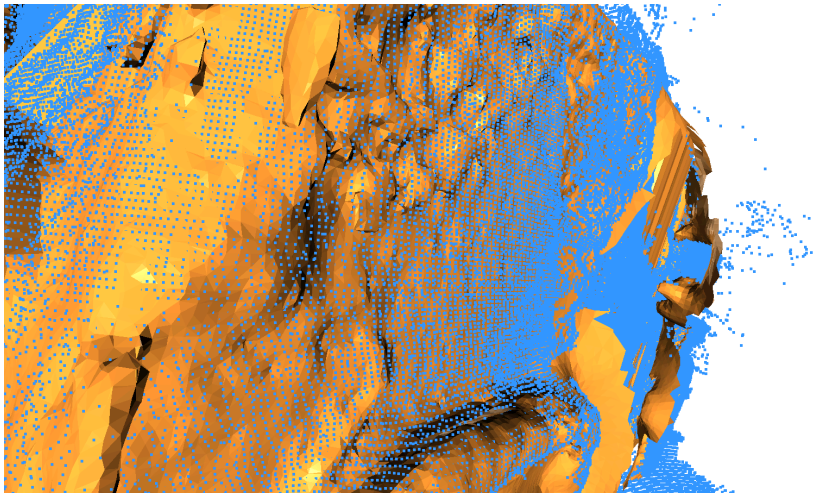


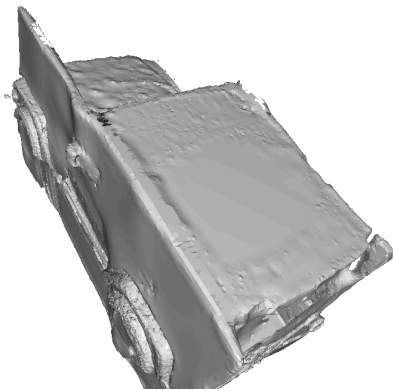
The Side Bulge Problem & Normal Orientation Root Cause

A persistent bulge artifact appeared on the vehicle sides near mirrors. The cause was concluded to be due to normal vector orientation. Inward pointing normals caused Poisson to interpret the surface boundary incorrectly, creating such bulges. Open3d's *orient_normals_consistent_tangent_plane()* with careful parameter selection fixed it.



This figure demonstrates a final mesh being compared to its original state, during a long period of time, I could not find the correct solution for fixing the bulges in the sides of the car. Later this was discovered that the normals that were created around the mirror were incorrect and pointed to the wrong angles which would create “bulges” on the sides of the car. Here is a close up of it in another test.





Post-Mesh Pipeline Experiments

There were many different attempts into achieving a detailed mesh that I was satisfied with. One approach that was attempted for a bit but later was scrapped was the idea of dual poisson blending. The idea was to blend a detailed poisson mesh with a fast one using per-vertex displacement, this failed for many reasons, one of them being that the two meshes had different resolutions so vertex counts did not match per vertex and that computation time took too long to try and solve the other issues.

Other smoothing attempts were used with Taubin smoothing, the mesh was too dense for smoothing to be correctly registered and the idea was also scrapped. This was a step in the right direction but it also came with its challenges. An idea to circumvent was to have pre mesh adaptive voxel thinning around the mirror zone and body at different centimeters but this failed as it completely tore the mesh. HC Laplacian smoothing with pinned boundary vertices had an improvement and so did the use of PymMeshLab bilateral smoothing + isotropic re-mesh. This came with its consequences as it had a massive shrinkage of triangles (from 50k to 14k)

The first breakthrough was using Decimation, it achieved low poly meshes (something that we needed) and with a few iterations a percentile based zones were used to help achieve a detailed look whilst still keeping the mesh as we wanted.



The PCD-First Paradigm Shift

In all of the previous stages these steps were being done with the already built Meshes, they were trying to fix mesh quality after Poisson reconstruction, but by that point Poisson had already baked in the point cloud's problems. The new strategy would be to clean and augment the cloud before meshing, so Poisson would get a better input from the start.

The first problem to tackle was the fact that the client required a curtain/skirt around the entire car, this was useful for us as we would not have to worry about wheel details as the acceptance would be for having the curtain cover the wheels.

It is made by scanning horizontal Z-slices top-down, looking for internal gaps in the X axis at the fender lip heights. When a gap is found, it trims the side zone selectively and extrudes curtain walls from the arch down to $z=0$ (0 being the floor of the scan). This was a step in the right direction and almost did everything, the only other problem is that there are times in the scans where walls were not identified correctly due to the lack of points, the idea was to compute a concave hull of the cars footprint and extrude it as side walls, it failed as the mirrors stick out from the body and would create gaps in the walls.

This was later fixed by doing many things. Firstly, it detects the mirror columns by identifying x positions where the surface protrudes over 10cm outward in the front 45% of the car. It interpolates behind mirrors by using SciPy's `interp1d` to reconstruct the body wall y values where mirrors occlude the door panel. It fills the remaining gaps by applying a *distance_transform_edt + gaussian_filter* that smooths over holes. Finally, it creates a split front/rear wall generation that processes the front and rear walls separately to avoid any ridging artefacts that stick out of the car that would also, in many cases create double layers.

Final pipeline

For the ease of simplicity, steps will be numbered with a short description of the final pipeline to not repeat information:

1. Voxel Downsampling - Grid-based density reduction at 0.008m resolution. Reduces point count while preserving spatial structure.
2. Normal Estimation - Hybrid KDTree search (radius=0.03m, max_nn=30). Computes per-point surface normals required by subsequent geometric filters.
3. RANSAC Plane Removal - Iteratively detects large planar surfaces (room walls, floor, ceiling) using RANSAC (1000 iterations, distance threshold 0.04m). Uses bounding-box inner/outer margin analysis to distinguish room walls from car body panels. Only removes planes meeting orientation criteria (Z-component < 0.3 for vertical planes, or $Z > 2.0\text{m}$ for ceiling).

4. Normal-Based Floor Filter - Removes remaining floor points missed by RANSAC. Points with normal Z-component > 0.85 and below 0.35m are classified as floor.

5. Z-Range Crop - Hard bounds $[-0.05\text{m}, 2.0\text{m}]$ on Z-axis. Eliminates floor/ceiling remnants entirely.

6. Statistical Outlier Removal - Gaussian-distribution-based filtering. For each point, computes mean distance to 10 nearest neighbors; removes points exceeding 3.3 standard deviations.

7. Radius Outlier Removal - Removes points with fewer than 5 neighbors within 0.05m radius. Eliminates isolated noise clusters that statistical filtering misses.

8. DBSCAN Clustering - Density-based spatial clustering ($\text{eps}=0.10\text{m}$, $\text{min_points}=20$). Keeps only the largest cluster, removing floating debris (e.g., foam particles, sensor artifacts).

9. Raw Recovery - Novel contribution. Uses the cleaned cloud as a spatial "guide" to recover original raw-scan detail. A cKDTree nearest-neighbor query ($\text{radius}=0.001\text{m}$) restores sub-millimeter surface detail that was lost during aggressive downsampling and filtering. Final post-recovery floor filter removes any residual floor points recovered during this step.

Output: $\sim 400\text{k}$ points from $\sim 1\text{-}2\text{M}$ input. Three data structures: cleaned cloud, guide cloud, and per-step statistics log.

Dynamic partitioning using data-driven percentile boundaries (no hardcoded thresholds):

- 10. Mirror zones (Y-axis):** 12th/88th percentile boundaries define left/right mirror regions
- Bumper zones (X-axis): 8th percentile from front/rear defines bumper extent
- Roof fill height: 97th percentile of Z-range
- Roof spike threshold: 99th percentile of Z-range

Region-specific voxel downsampling applies 4 different resolutions:

Removes points above the 99th Z-percentile that represent antenna tips or sensor noise above the roofline.

11. Detection: Projects top 12% of Z-range points to XY plane, creates 0.05m grid, computes occupied-cell coverage. If coverage $< 40\%$, a roof gap is detected.

Patching: Computes 2D convex hull of top-band points, uses Shapely to create boundary contour, fills interior with grid points at 0.03m spacing. All patch points placed at fill_z height with upward normals $[0, 0, 1]$.

12.

For front and rear bumpers:

1. Extract points in bumper zone
2. Create Y-Z grid at 0.015m resolution
3. Build depth map: per cell, store min(X) for front or max(X) for rear
4. Fill empty cells using Euclidean distance transform nearest-neighbor interpolation
5. Apply 0.005m inset offset (pushes surface inward)
6. Generate point grid with inward-facing normals

13.

algorithm to automatically find the curtain-Z height (below which side points are removed):

1. Extract body perimeter points via convex hull analysis
2. Build Z-histogram from roof to floor in 0.03m bands
3. Scan bands top-down for X-direction gaps (wheel arch signature)
4. When X-gap ratio exceeds 8% threshold, arch detected
5. Set curtain_z = arch_z + 0.10m margin
6. Fallback: 10th percentile of perimeter Z values if no arch found

14.

Detection algorithm:

1. For each side (left/right): extract strip of points near mirror boundary
2. Bin into X-direction cells (0.05m width)
3. Per bin: compute 80th percentile Y (body wall) and 5th percentile Y (mirror extreme)
4. Calculate protrusion = body_wall - extreme_Y
5. Mark bins as "mirror" if protrusion > 0.10m AND in front 45% of X-range
6. Interpolate body_wall_y across mirror bins from non-mirror neighbors

Output per side: body wall Y-coordinates, mirror mask, protrusion values, bounding boxes.

15.

Removes side zone points below curtain_z. Preserves front/rear zones (bumpers, lights). Eliminates mudflaps, undercarriage geometry.

16.

Depth-mapping approach:

1. Extract strip of points near mirror boundary, filter by height
2. Create X-Z grid (0.05m resolution)
3. Build depth map: Y-coordinate per cell (85th/15th percentile depending on side)
4. Fill NaN cells using EDT
5. Apply Gaussian smoothing (sigma=3.0)
6. Apply 0.015m wall offset (push inward)

7. Generate dense point grid with perpendicular normals

Same depth-map approach as side walls but oriented on Y-Z grid with X-depth values.

17.

1. Extract body perimeter points above curtain_z
2. Project to XY, compute concave hull (Shapely, alpha ratio=0.3)
3. Optional inset by 0.03m
4. Generate grid at 0.04m spacing inside footprint
5. Floor points at z_floor with downward normals [0, 0, -1]

18.

Combines: thinned car cloud + roof patch + bumper patches + side walls + front/rear walls + floor cap into a single assembled point cloud ready for meshing.

19.

Screened Poisson Reconstruction.

1. Normal Estimation - Hybrid KDTree (radius=0.06m, max_nn=15)
2. Normal Orientation - Consistent tangent-plane method (k=30 neighbors), ensures outward-facing normals
3. Screened Poisson Reconstruction - Octree depth=6, scale=1.3, quadratic basis. Produces triangle mesh + per-vertex density values
4. Density Trimming - Removes vertices below 0.2th density percentile (noise artifacts)
5. Bounding-Box Crop - Removes vertices 0.02m outside original cloud extent
6. Hole Filling - Detects boundary edges, traces connected loops via adjacency graph, triangulates loops < 500 edges
7. Post-Processing - Remove degenerate triangles, non-manifold edges.

20.

1. Mirror Vertex Snapping- Restores original scan detail in mirror regions. Uses cKDTree to snap mesh vertices inside mirror bounding boxes to nearest raw scan point within 0.02m. Re-computes normals post-snap. Result: mirrors match original scan, body remains smooth Poisson surface.

2. Watertight Closure - PyMeshFix algorithm: detects non-manifold edges, fills holes, removes self-intersections, ensures closed consistent orientation.

3. Mesh Orientation Validation (r03)

4. STL Export Applies meters-to-millimeters scaling (x1000). Outputs final watertight mesh.

Entire pipe line :



Using AI Foundation Models to Recognize Car Parts in Point Clouds

1. Problem Context

Our current approach uses:

- Geometric heuristics (bounding box positions)
- Height thresholds
- Clustering
- Statistical filtering

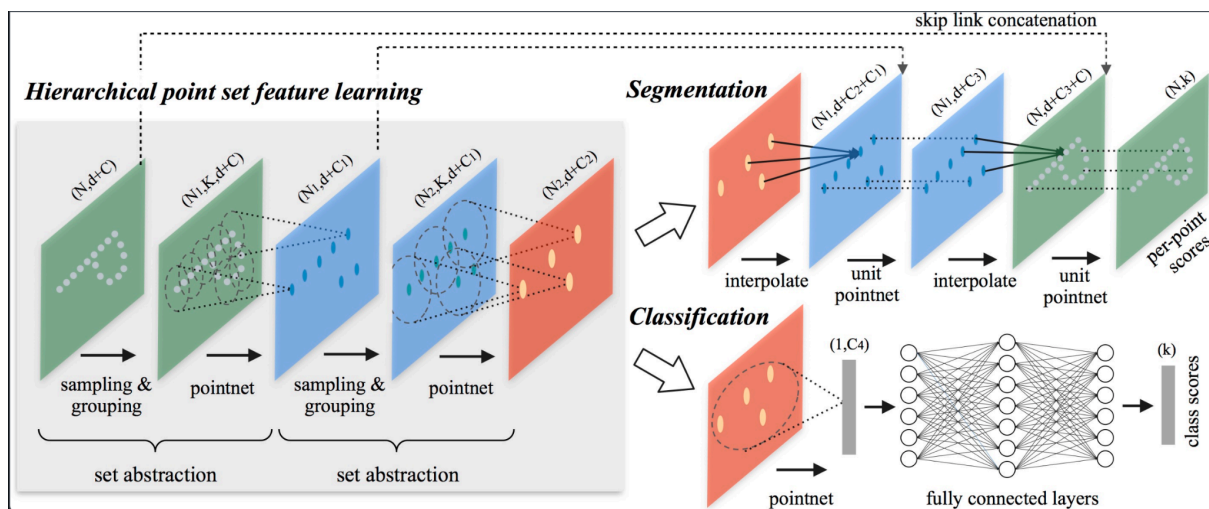
This comes with certain benefits such as time, it's pretty fast, explainable and it works well with little data. But also it has some disadvantages: hard to generalize, breaks on unusual cars and needs manual tuning.

AI recognition could help improve robustness. There are 2 realistic categories that could be used given that we have a time constraint, we will focus on PointNet and PointNet++..

2. Pretrained 3D Deep Learning Models

PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space

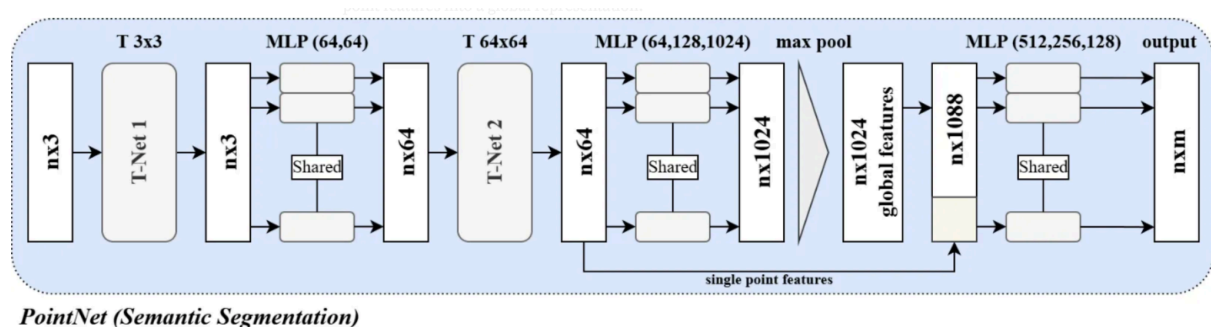
Created by Charles R. Qi, Li (Eric) Yi, Hao Su, Leonidas J. Guibas from Stanford University.



PointNet (the v1 model) either transforms features of *individual points* independently or process global features of the *entire point set*. However, in many cases there are well defined distance metrics such as Euclidean distance for 3D point clouds collected by 3D sensors or geodesic distance for manifolds like isometric shape surfaces. In PointNet++ we want to respect *spatial localities* of those point sets. PointNet++ learns hierarchical features with increasing scales of contexts, just like that in convolutional neural networks. Besides, we also observe one challenge that is not present in convents (with images) -- non-uniform densities in natural point clouds. To deal with those non-uniform densities, we further propose special layers that are able to intelligently aggregate information from different scales.

We could use PointNet++ for **Shape Classification**, classify ModelNet40 shapes (using point clouds with XYZ coordinates). It can be used for **Object Part Segmentation** as well.

PointNet:



To achieve permutation invariance, PointNet uses a simple and powerful strategy:

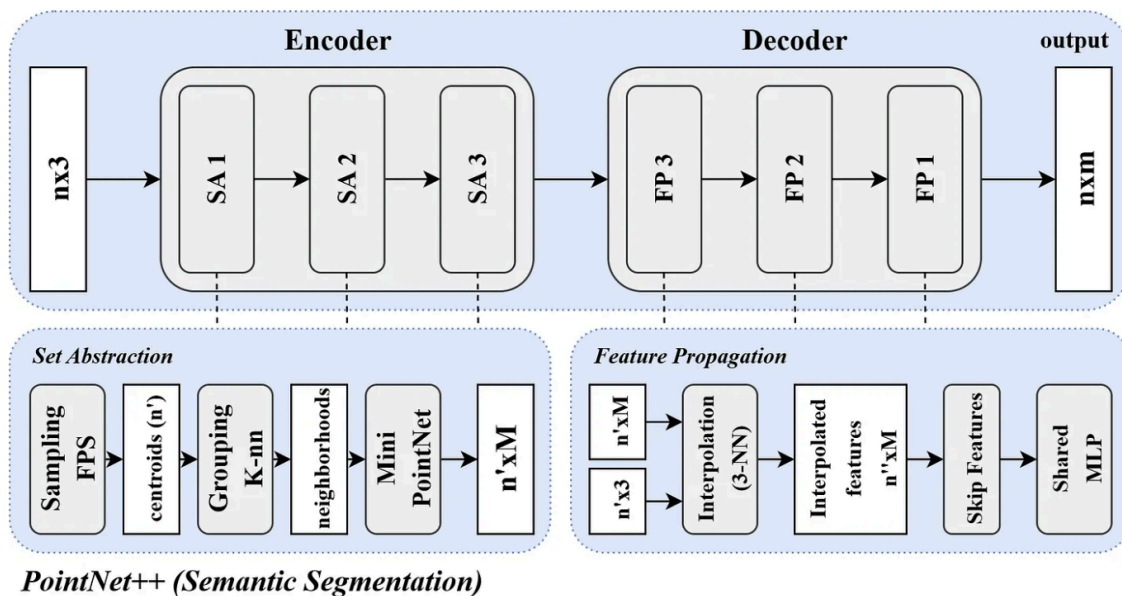
- **Shared Multi-Layer Perceptrons (MLPs):** Each point in the cloud is processed independently and identically through a series of shared MLPs. This ensures that the network learns to extract the same type of features for every point, regardless of its position in the input order.
- **Symmetric Function:** A symmetric function, specifically max pooling, is applied across the feature dimension of all points. This function aggregates the learned features from all points into a single global feature vector that describes the entire shape. This aggregation is inherently invariant to the order of the points.
- **Transformation Networks (T-Nets):** To ensure invariance to geometric transformations, PointNet incorporates small neural networks called T-Nets. These networks predict an affine transformation matrix that is

applied to the input points and their features, canonicalizing the data before feature extraction.

The key insight of PointNet is to learn spatial encoding for each point individually and then aggregate all individual point features into a global representation.

PointNet++

While PointNet was revolutionary, it did not explicitly capture the local geometric structure induced by the metric space of the points. It treated each point independently before global aggregation, missing fine-grained patterns in local regions. PointNet++ was proposed to address this limitation by introducing a hierarchical feature learning approach that captures patterns at multiple spatial scales.



Block diagram of the PointNet architecture

Encoder

The architecture of PointNet++ is built upon a series of Set Abstraction (SA) modules, which progressively abstract larger regions of the point cloud into high-dimensional feature vectors. Each SA module is composed of three key stages:

- **Sampling Layer:** A subset of points is selected from the input to define the centroids of local regions. The algorithm used is *Farthest Point Sampling (FPS)*, which ensures that the centroids are well-distributed

throughout the point cloud, providing a better spatial coverage than random sampling.

- **Grouping Layer:** For each centroid, a local neighborhood is defined by collecting nearby points within a certain radius or using k-nearest neighbors. These points form a small cluster that captures local geometric information.
- **PointNet Layer:** A mini-PointNet network is applied to each local group to extract a high-level feature vector summarizing the geometric structure of that neighborhood.

By stacking several SA modules, PointNet++ builds an encoding hierarchy: at each level, the number of points decreases while the complexity of the features increases. This process is analogous to convolution and pooling layers in 2D CNNs, which reduce spatial resolution while enriching the representation.

Decoder

For dense prediction tasks such as semantic segmentation, the model must recover per-point predictions for the original resolution. PointNet++ handles this through a **hierarchical feature propagation decoder**, using interpolation and skip connections to transfer and refine features level by level.

- **Interpolation:** During decoding, features from higher-level representations (with fewer points and more global context) are transferred back to denser levels via interpolation. The interpolation is weighted inversely by distance to nearby points.
- **Skip Connections:** Interpolated features are combined with those preserved from the encoder at the same level. This fusion helps recover fine local details that might have been lost during subsampling, while maintaining access to global context.
- **Shared MLP:** After merging, the resulting features are refined through a small shared MLP to update the point-wise representations. This is repeated until all original points receive a final feature vector.

This hierarchical encoder-decoder structure enables PointNet++ to learn robust and detailed representations at multiple scales, achieving superior performance in complex 3D scene understanding tasks.

In our case for car wash we could potentially use PointNet for detection of wheels, mirrors, roof and body of the car.

Advantages of PointNet:

- Made for point clouds
- High accuracy
- Industry standard
- Works directly on XYZ data

Limitations:

- Requires training data
- Training takes weeks
- Requires GPU
- Integration complexity

To integrate this approach to our case, it would take more than 6 weeks since we don't have sufficient amount of data to work with properly.